FUNDACIÓN ARGENTINA DE SMALLTALK · 12 DE NOVIEMBRE DE 2026

protoST por dentro

Inmutabilidad, un GC concurrente sin write barrier, una VM sin contextos y actores sobre hilos nativos. Charla y laboratorio.

protoST 0.9.0 · protoCore 3.5

github.com/gamarino/protoST

LAS TRES HORAS

Plantear, discutir, probar en sus máquinas

Hora	Bloque	Qué se ve
0:00	Apertura	protoST hoy, la tesis
0:10	Parte I — el modelo	Identidad y valor, estructuras, GC, decisiones, módulos
0:55	Laboratorio 1	Ejercicios 1 a 4 en sus máquinas
1:15	Corte	10 minutos
1:25	Parte II — la VM	Bytecode, envíos, retorno no local, excepciones, interoperación, actores
2:15	Laboratorio 2	Ejercicios 5 a 10
2:50	Cierre	Desafíos abiertos y preguntas

UNA APUESTA, NO UNA CONCLUSIÓN

Si todo valor es inmutable, el estado compartido deja de romperse y el paralelismo real se vuelve barato de razonar.

Les muestro qué compra esa decisión, cuánto cuesta y qué deja afuera. Hasta dónde llega, no lo sé: para eso vine.

POR QUÉ CON USTEDES

No vengo a enseñarles una VM: vengo a mostrarles una apuesta y a pedirles criterio

Lo que traigo

- un núcleo de objetos inmutables con hilos nativos
- un Smalltalk que corre encima, con sus costos medidos
- bugs encontrados y arreglados, cada uno con su test
- muchas preguntas que no sé contestar

Lo que tienen ustedes

- años construyendo y manteniendo VMs de Smalltalk
- contextos, become:, imagen y depuración viva resueltos de verdad
- el criterio para saber qué de esto vale la pena

Cada parte termina con preguntas abiertas. Las respuestas que me importan son las suyas.

DESDE LA CHARLA DEL 15/10

De 0.7.0 a 0.9.0 en cuatro semanas

8,9× → 2,15×

fib frente a CPython 3.14: el camino de envío y los bloques inlineados que ya no capturan

__st_send__

otro runtime le manda mensajes Smalltalk a un objeto protoST, como un envío real

#[1 2 255]

0.9.0: ByteArray inmutable, archivos binarios, MessagePack y deadlines en los futures

VM.md

la VM documentada, 43 opcodes, --disasm, y cinco bugs encontrados al escribirla, corregidos

DAP

depuración en VS Code: breakpoints de excepción, varios hilos, logpoints, depuración remota

1291 tests

en Linux, macOS y Windows, y un job de ThreadSanitizer; programas propios, no la suite ANSI

Fuentes: CHANGELOG.md (0.7.0 a 0.9.0), docs/STATUS.md · benchmarks/reports/2026-10-20-send-path-inline-captures.md · 5

PARTE I · 45 MINUTOS

El modelo

Identidad y valor · escribir es publicar · estructuras persistentes · un GC concurrente sin write barrier · las decisiones que esto obliga · módulos

IDENTIDAD, ESTADO, VALOR

Un objeto mutable es una identidad que apunta a una instantánea inmutable

Identidad

la celda handle, con un mutable_ref que no cambia nunca

`ref = 17`

Tabla global

256 shards; cada raíz es un atomic a una lista persistente ref → estado

`shard = ref % 256`

Valor

el estado actual: un objeto inmutable con sus atributos en un AVL

`v1 → v2 → v3`

Leer resuelve una instantánea: todos los atributos de ese objeto son consistentes entre sí. Escribir publica una instantánea nueva con un compare-and-swap.

LA UNIDAD DE MEMORIA

Todo es una celda de 64 bytes o una palabra con tag

La celda

8 B · puntero a vtable de C++

8 B · next_and_flags: bit de marca + enlace

48 B · carga útil. Un objeto: padres, atributos (AVL), mutable_ref

Sin clases de tamaño ni objetos de tamaño variable: todo es un árbol de celdas. Los bytes crudos viven fuera, en buffers externos.

La palabra

Los 6 bits bajos son el tag (las celdas están alineadas a 64). En el tag 1 viven los inmediatos:

- SmallInteger de 54 bits con signo
- Character, Boolean, nil
- Strings de hasta 6 bytes, en la palabra misma

Los Float van en una celda: no hay NaN-boxing.

EL CAMINO DE ESCRITURA

Escribir una variable de instancia es publicar una versión

1. Resolver el estado actual (caché por hilo o AVL)
2. Copiar el camino del AVL de atributos
3. Una celda nueva para el estado
4. Copiar el camino de la raíz del shard
5. compare_exchange sobre esa raíz; si falla, reintentar desde 1

costo por escritura

$$\log_2(\text{atributos}) + \log_2(\text{mutables} / 256) + 1 \text{ celdas de } 64 \text{ B} + 1 \text{ CAS}$$

Cada versión superada es basura. El reintento no tiene tope: un tope anterior perdía escrituras.

QUÉ GARANTIZA Y QUÉ NO

Ninguna escritura se rompe, pero un incremento se puede perder

Garantiza

- cada lectura ve un estado entero del objeto
- una racha de asignaciones se publica junta (IVAR_GROUP)
- pasar un objeto a otro hilo es pasar un puntero

No garantiza

- leer-modificar-escribir atómico: n := n + 1
- consistencia entre dos objetos

Ejercicio 2 · 8 actores × 100 000 incr

objeto compartido

140 634

de 800 000, en una corrida

actor · Atom

exacto, siempre

ESTRUCTURAS PERSISTENTES

Copiar es O(1); modificar copia un camino

Estructura	Forma	En protoST
ProtoList	AVL, un elemento por celda; inline hasta 5	el contenido de Array y OrderedCollection
ProtoSparseList	AVL por clave entera; inline hasta 3 pares	atributos, variables de instancia, la tabla de mutables
ProtoString	inline ≤ 6 bytes · símbolo perenne · cuerda AVL	String inmutable; Symbol
ProtoTuple	cuerda de nodos de 4, internada, perenne	constantes y claves
ProtoMap	hash con buckets y semántica de claves del lenguaje	Dictionary, Set

Ejercicio 1: copy de un Array de 1 M elementos, 0 ms; cada at:put: sobre la copia, unos 1,4 µs.

El precio: 64 B por elemento y unas n·log n celdas transitorias al construir en masa (19:1 con 100 k).

EL RECOLECTOR

Mark-sweep que no mueve: la pausa solo toma raíces

Pausa
quórum de hilos estacionados; raíces precisas; copia de las 256 raíces de mutables

Marcado
concurrente, un hilo, pila acotada de 8 MiB con re-escaneo por desborde

Barrido
concurrente, 8 cursores en paralelo, hilos auxiliares mientras rinden

Limpieza
mutables muertos; desmarcar

Generación implícita por contexto

Lo que asigna un contexto queda en su cadena joven y no es candidato hasta que el contexto muere o pasa 10 000 asignaciones. No hay generación vieja: cada ciclo vuelve a marcar todo lo vivo.

Raíces precisas, sin escanear el stack de C

Los frames de protoST son slots en un arreglo por hilo que el GC recorre. Ningún objeto puede vivir solo en una variable de C++ a través de una asignación.

LA PIEZA QUE COMPRA LA INMUTABILIDAD

Marcado concurrente sin write barrier: SATB por instantánea, no por barrera

Campos const

Cada processReferences lee solo campos fijados al construir la celda. Un objeto ya visto no cambia por debajo del marcador.

2 KB de mutabilidad

Toda la mutabilidad del proceso son 256 raíces atómicas. La pausa las copia: esa copia es el snapshot-at-the-beginning.

Lo nuevo es joven

Lo asignado después de la pausa no es candidato. La basura flotante dura a lo sumo un ciclo.

Desde protoCore 3.1.0 los estados no son raíces: un estado se marca solo a través de su handle, así que un ciclo de mutables inalcanzables se libera en un ciclo.

MEDIDO

La pausa no es el costo

≤ 0,23 ms

stop-the-world por ciclo, 41 corridas con hasta 12 hilos asignando

21–270 ms

espera del quórum antes de la pausa, por corrida: techo fijo / controlador adaptativo

41–99 %

del tiempo del recolector se va en el barrido; con 12 hilos, un marcador es el cuello

En protoST: pausas de hasta 0,27 ms en los benchmarks; antes de 0.6.2, 20–31 ms porque cada pausa recorría un arreglo de frames de 1 M entradas por hilo (K6).

No publicado todavía: una distribución de pausas (p99, p999). Todo se midió en una notebook Ryzen 5 5500U.

protoCore: docs/GarbageCollector.md, informe 2026-10-03-gc-phase-breakdown · protoST: KNOWN_ISSUES.md K6 · 14

CUÁNDO RECOLECTAR

Un controlador adaptativo decide el tamaño del heap

- **H**, el presupuesto: 75 % de la memoria física o del límite del proceso
- **S**, el límite blando: arranca en 128 MiB y solo crece mientras el programa espera al recolector
- arranca cada ciclo antes del techo, según la tasa de asignación medida
- sin límite y sin controlador, ningún ciclo arranca solo

para el laboratorio

```
PROTOCORE_HEAP_TRACE=1
una línea por ciclo: L, S, H, C, stw
PROTOCORE_HEAP_LIMIT_CELLS=2000000
techo fijo: 2 M celdas = 128 MB
PROTOCORE_ADAPTIVE_HEAP=0
el techo de 640 MB de 0.6.1
```

Desde 0.6.2 es el default de protoST: actores 16–43 % más rápidos con entre un quinto y la mitad de la memoria. El precio: el pico ya no está acotado (un bucle que asigna sin parar llegó a 0,88–3,5 GB).

HILOS

Hilos nativos sin lock global; el GC necesita que todos lleguen

Asignar no toma locks

Cada ProtoThread tiene su lista libre privada; recargarla sí pasa por un mutex global.

Safepoints cooperativos

Una bandera relajada cada 64 asignaciones; protoST también en cada llamada y en cada salto hacia atrás.

E/S bloqueante: salir del mundo

UnmanagedScope cuenta al hilo como estacionado mientras dura la llamada: 51–70 ns en protoCore 3.3.1.

El contrato que muerde

Una llamada bloqueante sin ese paréntesis deja al quórum esperando para siempre: el GC se cuelga.

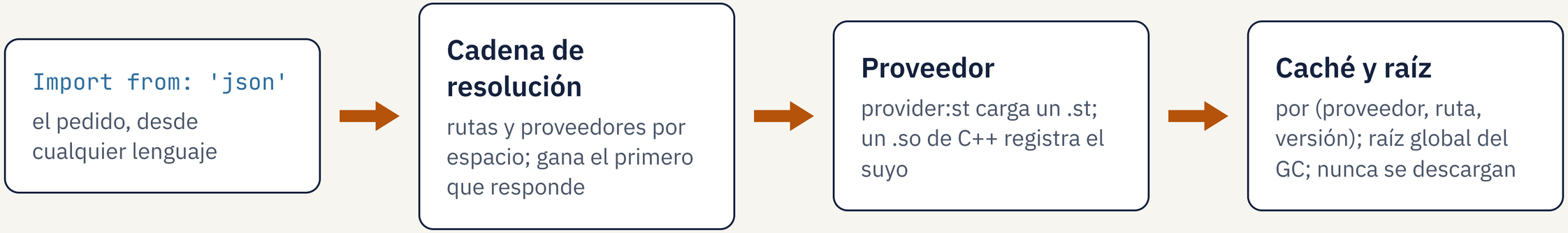
LO QUE EL NÚCLEO LE IMPONE A SMALLTALK

Cinco decisiones, cada una con su causa

En protoST	Por qué	Lo que cuesta
Clases = prototipos con nombre	protoCore es de prototipos con varios padres	metaclase delgada; uses: sin resolución de conflictos
Strings inmutables (D34)	un String es una cuerda de protoCore, sin identidad mutable	at:put: da ModificationForbidden
#foo == 'foo' (D35)	≤ 6 bytes van en la palabra; no queda lugar para el tag de símbolo	#foo class es String; isSymbol false
Cadena de padres fija (D21)	búsqueda sin asignar y estable por instantánea	addBehavior: solo alcanza a instancias nuevas
Actores, no Process	hilos nativos sin lock global	sin Semaphore, Delay ni fork

MÓDULOS SIN IMAGEN

UMD: una sola forma de encontrar módulos para todos los lenguajes



En protoST un .st es un módulo: corre una vez, exporta sus clases salvo las que empiezan con _, y se busca en el directorio actual, $STPATH, el venv y lib/.

Hoy el protost instalado solo tiene provider:st: cargar un módulo de otro lenguaje necesita un host que registre su proveedor.

PARA DISCUTIR · EL MODELO

1. ¿Vale pagar una asignación en cada store a cambio de no tener write barrier ni locks?
2. ¿Qué idioms de Smalltalk dependen de una identidad mutable que este modelo rompe?
3. ¿Cómo harían una generación vieja sin barrera de escritura?
4. ¿Qué tendría que medir este GC para que lo tomen en serio?

No tengo respuesta cerrada para ninguna. Después, el laboratorio 1 pone a prueba la 1 y la 3.

LABORATORIO 1 · 20 MINUTOS

Predigan, corran, discutan

`01-copiar-es-gratis.st`

copy O(1), at:put: O(log n), alias, String

`02-actualizaciones-perdidas.st`

objeto compartido, actor, Atom

`03-forma-de-clase.st`

instancias viejas frente a >>, addBehavior: y redeclarar

`04-gc-en-vivo.st`

PROTOCORE_HEAP_TRACE=1: ciclos, pausas, cuatro actores

docs/talks/2026-11-12-fas-workshop/ejercicios · ./verificar.sh compara con la salida esperada

PARTE II · 50 MINUTOS

La VM

Bytecode y frames · bloques sobre objetos inmutables · el envío · retorno no local · excepciones · lo que falta · interoperación · actores

DE ARCHIVO A EJECUCIÓN

Un archivo es una unidad de compilación; declarar es enviar



Adders >> adderFor: n compila a

```
PUSH_GLOBAL     Adders
PUSH_BLOCK      main.0
PUSH_CONST      #adderFor:
SEND_KEYWORD    #__installMethod:as:
```

Una clase es **__subclassNamed:** + **STORE_GLOBAL**; un método, una clausura sobre su propio módulo, instalada con un envío. El kernel (lib/kernel/*.st) se carga igual antes del script.

BYTECODE

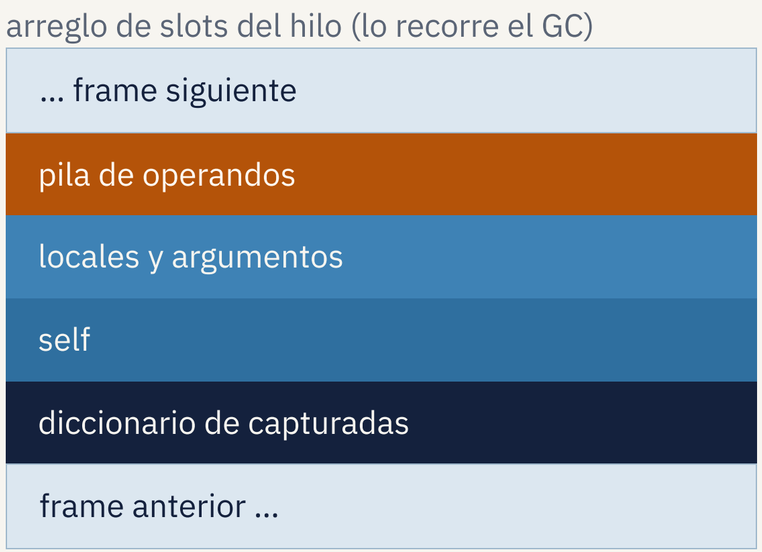
43 opcodes en palabras de 2 bytes

Grupo	Opcodes
Valores y pila	PUSH_CONST · PUSH_NIL/TRUE/FALSE · PUSH_SELF · MAKE_ARRAY · PUSH/STORE_LOCAL · DUP · POP
Variables	PUSH/STORE_GLOBAL · PUSH/STORE_INSTVAR · PUSH_OWN_INSTVAR · STORE_CLASSVAR · IVAR_GROUP(_END)
Capturadas	MAKE_CAPTURED · DEFINE_CAPTURED · PUSH_CAPTURED · STORE_CAPTURED
Control	JUMP · JUMP_IF_TRUE/FALSE · JUMP_BACK · ASSERT_BOOL_OR_DNU
Envíos	SEND_UNARY/BINARY/KEYWORD · SEND_SUPER · SEND_CALL · RETURN · RETURN_TOP · PUSH_BLOCK
Atajos enteros	BIN_INT_ADD/SUB/LE/LT/GE/GT/EQ; si no son SmallInteger o desborda, sigue como SEND_BINARY

Opcode + operando de 8 bits; hasta dos prefijos EXTEND llevan el operando a 24 bits. No hay opcodes de excepciones ni de colecciones, salvo MAKE_ARRAY.

ACTIVACIONES

Un frame no es un objeto: es un rango de slots



- el engine no es recursivo: un vector de frames que solo guardan enteros
- un solo ProtoContext por hilo, ninguno por activación
- lo que se desapila sigue en el arreglo y queda enraizado
- hasta 2^20 slots: unas 19 000 activaciones, con un Error capturable al pasarse
- una primitiva que corre un bloque (do:, ensure:) anida otro engine: hasta 1000

EJERCICIO 5 · PROTOST --DISASM

El mismo bucle, inlineado y como bloque

`1 to: n do: [:i | s := s + i]`

```
9  PUSH_LOCAL     3  ; i
10  PUSH_LOCAL     4  ; end
11  BIN_INT_LE     2  ; #<=
12  JUMP_IF_FALSE  11 ; -> 24
13  PUSH_LOCAL     2  ; s
14  PUSH_LOCAL     3  ; i
15  BIN_INT_ADD    3  ; #+
16  DUP
17  STORE_LOCAL    2  ; s
…
23  JUMP_BACK      15 ; -> 9
```

`(1 to: n) do: [:i | s := s + i]`

```
0  MAKE_CAPTURED  0  ; root dict
2  DEFINE_CAPTURED 0  ; s
9  SEND_KEYWORD   3  ; #to:
10  PUSH_BLOCK     0
11  SEND_KEYWORD   4  ; #do:

[] in sumaConBloque:
0  PUSH_CAPTURED  0  ; s
1  PUSH_LOCAL     0  ; i
2  BIN_INT_ADD    1  ; #+
4  STORE_CAPTURED 0  ; s
```

Inlineado: s vive en el frame y el bucle no envía nada. Como bloque: s pasa a un diccionario de capturadas y cada vuelta publica una versión.

La cadena de ámbitos es una cadena de prototipos

```
contadores
  ^ (1 to: 3) collect: [:i | | c |
      c := i * 10. [ c := c + 1 ] ]
```

```
0  MAKE_CAPTURED   1 ; child of the
                       inherited dict
2  DEFINE_CAPTURED 0 ; c
5  SEND_BINARY     2 ; #*
7  STORE_CAPTURED  0 ; c
9  PUSH_BLOCK      0
```

- las variables capturadas son atributos de un objeto **mutable**
- cada activación de bloque abre un hijo del diccionario de afuera
- buscar una variable es buscar un atributo por la cadena
- una clausura: __bc_ptr__, __captured__, __home_frame__, __block_self__

Los tres bloques dan 13, 21 y 31: cada uno tiene su c.

EL ENVÍO

Sin inline caches: la caché es de protoCore y se invalida sola

1. ¿Proxy de actor? Encolar y devolver un Future
2. ¿Bloque y value… con la aridad justa? Frame en el mismo engine
3. Buscar el selector por la cadena de prototipos
4. No está: doesNotUnderstand:
5. Método: apilar frame y safepoint
6. Atributo valor: un unario lo devuelve (así se leen los módulos)
7. Primitiva: índice en un registro, con traducción de excepciones

caché por hilo, 1024 entradas

```
(instantánea, símbolo)
  → ¿es atributo propio?
```

Hechos positivos y negativos por eslabón de la cadena. Cambiar una clase publica otra instantánea: las entradas viejas ya no coinciden. Se vacía en cada pausa del GC.

UNA GARANTÍA QUE SMALLTALK-80 NO DA

Una racha de asignaciones se publica de una sola vez

```
x: ax y: ay
  x := ax.
  y := ay.
  z := 0
```

```
0  PUSH_LOCAL      1 ; ax
1  IVAR_GROUP      0 ; x, write 1 of 3
2  PUSH_LOCAL      2 ; ay
3  IVAR_GROUP      1 ; y, write 2 of 3
4  PUSH_CONST      2 ; 0
5  IVAR_GROUP_END  2 ; z, write 3 of 3
```

- de 2 a 16 asignaciones seguidas cuyos valores no pueden correr código
- una sola llamada a setAttributes: otro hilo ve las tres o ninguna
- un fallo del atajo entero en medio publica lo escrito hasta ahí
- el depurador las apaga; PROTOST_IVAR_GROUPS=off también

CLASES SOBRE PROTOTIPOS

super, la metaclase y la herencia múltiple

super por identidad

Al instalar un método, su bytecode recibe el __class_id__ de la clase. En ejecución se recorre la cadena del receptor buscando ese id y se sigue por sus padres.

Metaclase delgada

Los métodos de clase viven en el objeto clase con una marca. 3 class class es SmallInteger class, pero Metaclass, Behavior y ClassDescription no existen.

uses:

Más padres, en profundidad y de izquierda a derecha; gana el primero. No son traits de Pharo: sin conflictos, alias ni exclusiones.

Hasta 0.7.0 comparaba nombres: con m Figura subclass: #Figura, el super del módulo encontraba la Figura local y recursaba hasta "stack depth exceeded". Apareció preparando esta charla (S33).

RETORNO NO LOCAL SIN CONTEXTOS

Un ^ en un bloque: tres casos y uno que sorprende

Dónde está el home	Qué hace el engine
en el mismo engine	desapila frames hasta el home, inclusive
en un engine exterior (do:)	lanza NonLocalReturn de C++; los ensure: corren al desenrollar
ya retornó	BlockCannotReturn, capturable
vivo, pero en otro hilo	BlockCannotReturn también: el ^ no cruza hilos

¿El home está vivo? Se pregunta a un registro thread-local de engines. Un bloque con ^ mandado a otro actor no encuentra su home aunque el método esté esperando la respuesta. ¿Debería cruzar?

EXCEPCIONES

Buscar el handler sin desenrollar, desenrollar con C++

- on:do: es una primitiva: apila una entrada en una pila de handlers por hilo y corre el bloque en un engine anidado
- signal busca la entrada más nueva que coincide y corre el handler **encima** de los frames que señalaron
- resume: vuelve desde signal; return:, retry y caer al final desenrollan con excepciones de C++

17,3×

lo que tarda CPython en el benchmark de latencia de excepciones: el precio del desenrollado de C++

El premio

el depurador se detiene en el signal antes de que corra cualquier handler

Lo que falta: outer se comporta como pass (D7). Las excepciones de otros lenguajes llegan con su valor y no son reanudables.

LO QUE NO HAY, Y POR QUÉ

Algunas faltas son del modelo; otras, solo trabajo pendiente

Por el modelo

- **become:** la identidad es una entrada en la tabla, y un objeto inmutable no se puede cambiar por otro
- **thisContext:** los frames son enteros sobre slots; no hay contexto que reificar
- **Process, Semaphore, fork:** la concurrencia son actores sobre hilos nativos
- **imagen:** el programa es el archivo

Pendiente

- **outer** se comporta como pass (D7)
- IdentityDictionary e IdentitySet (ByteArray y #[ ] llegaron en 0.9.0)
- ScaledDecimal
- Metaclass, Behavior, True y False como clases
- preempción, buzones acotados y supervisión de actores

RENDIMIENTO EN UN HILO

Más lento que CPython en un hilo, y sin JIT

Benchmark	Proceso completo	Solo el trabajo	Proceso, antes
str_concat	0,92×	2,4×	0,95×
range_iterate	1,03×	2,3×	2,24×
attr_lookup	1,28×	2,3×	4,16×
int_sum_loop	1,41×	4,7×	1,41×
list_append	1,51×	12,3×	1,66×
fib	2,15×	4,9×	8,91×
exception_latency	17,29×	42,5×	20,65×

Tiempo de protoST ÷ tiempo de CPython 3.14 (más de 1: protoST más lento). Media geométrica de proceso completo: 1,92× (antes 3,25×). Arranque: 25 ms frente a 31 ms. No hay columna de Pharo, Cuis ni VAST: no lo medimos.

PARA DISCUTIR · LA VM

1. ¿Qué mínimo de thisContext necesita un depurador Smalltalk de verdad? ¿Alcanza con materializar frames bajo demanda?
2. ¿Dónde pondrían un inline cache si cada escritura publica una instantánea nueva?
3. ¿Desenrollar con excepciones de C++, o un mecanismo propio?
4. Sin imagen, ¿qué parte del entorno vivo se puede recuperar?

PARTE II · CONTINUACIÓN

Otros lenguajes y actores

Un objeto, dos lenguajes, ninguna copia · qué funciona hoy · actores, futures y la espera cooperativa · tres herramientas de concurrencia · cuánto escala

INTEROPERACIÓN

Dos lenguajes ven la misma celda: no hay copia ni serialización

Scala → Smalltalk

```
import st.bank.Account
val a = Account.`new`()
a.transfer_to(30, b)
```

llega como transfer:to: por __st_send__: un envío real, con doesNotUnderstand: y primitivas

Smalltalk → otro lenguaje

```
aFunction apply(balance)
geo circleArea(2)
```

envío call-form: nombre llano y argumentos posicionales y con nombre, como los métodos de protoCore

Excepciones

```
[ geo circleArea(-1) ]
  on: geo NegativeRadius
  do: [:e | ... ]
```

una sola ProtoException; el handler elige por la cadena de prototipos del valor

Enteros, Floats, strings, booleanos y nil son los mismos bits en todos los lenguajes. Las colecciones no: cada runtime tiene su envoltorio, y se usa el protocolo del otro o se copia explícitamente.

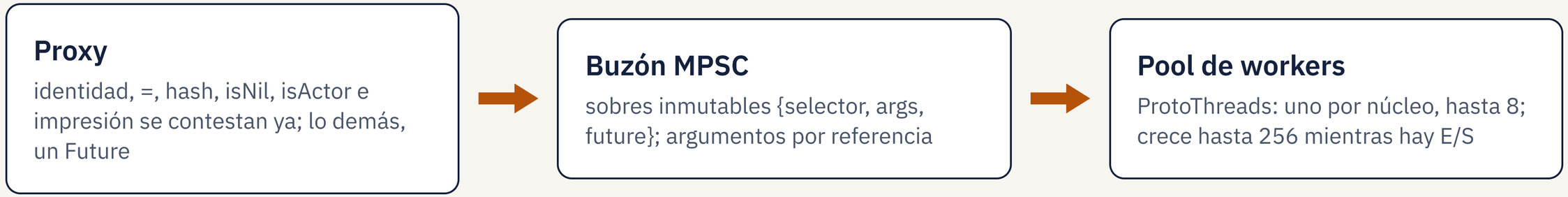
QUÉ FUNCIONA HOY

Funciona dentro de un host de prueba; los binarios instalados todavía no se cargan entre sí

Con	Estado	Dónde verlo
protoScala	los dos sentidos, clausuras y excepciones, en un mismo ProtoSpace	protoScala: tests/unit/protost_interop.cpp
Módulo C++	call-form y sus tipos de error, con un host que registra el proveedor	tests/umd/, protost_umd_host
protoJS, protoPython	sin proveedor alcanzable ni tests	—
ProtoClojure	"todavía no", dicho en su README	ProtoClojure/README.md
protost instalado	solo provider:st; Import from: 'x.scala' da module not found	probado con 0.9.0

ACTORES

Cualquier objeto se vuelve actor con asActor



Un mensaje por vez. Adentro, self es el objeto común: el estado son variables de instancia sin locks. Tres bandas de prioridad.

El lock es del proxy, no del objeto. Dos proxies sobre el mismo objeto, o una referencia anterior a asActor, se saltean el buzón.

ESPERAR UN FUTURE

wait tiene tres caminos según quién espera

Hilo principal

El script o el REPL bloquean el hilo del sistema en un semáforo por future, sin girar. Son clientes síncronos del mundo de actores.

Actor, desde bytecode

El engine guarda su vector de frames como objetos (pc, pila, locales, capturadas) y libera el worker. Al resolverse, el actor sigue en cualquier worker.

Actor, bajo una primitiva

Dentro de do: o ensure: hay frames de C++ que no se pueden guardar: el worker ayuda, corriendo mensajes de otros actores mientras espera (hasta 16 niveles).

El segundo camino es una continuación de un disparo: mil actores esperando corren sobre dos hilos (benchmarks/actors/cooperative_yield.st con PROTOST_WORKERS=2).

EJERCICIO 8

Un actor que espera no atiende a nadie

```
Par >> ping  ^ otro pong wait
Par >> pong  ^ otro valor wait

a ping wait
```

```
deadlock: this actor waits for a reply
that can only come from an actor
waiting on it …
```

- antes de estacionarse, wait sigue la cadena future → actor → lo que ese actor espera, hasta 256 pasos
- si vuelve a sí mismo: Error capturable, no un cuelgue
- una barrera seq_cst asegura que de dos actores que cierran el ciclo a la vez, al menos uno lo vea
- un ciclo que pasa por un Future new resuelto a mano no se detecta

Es el precio del estado sin locks (D37). Adentro de actores: thenDo:, whenAll:, whenAny:.

CONCURRENCIA

Tres herramientas en lugar de Process y Semaphore

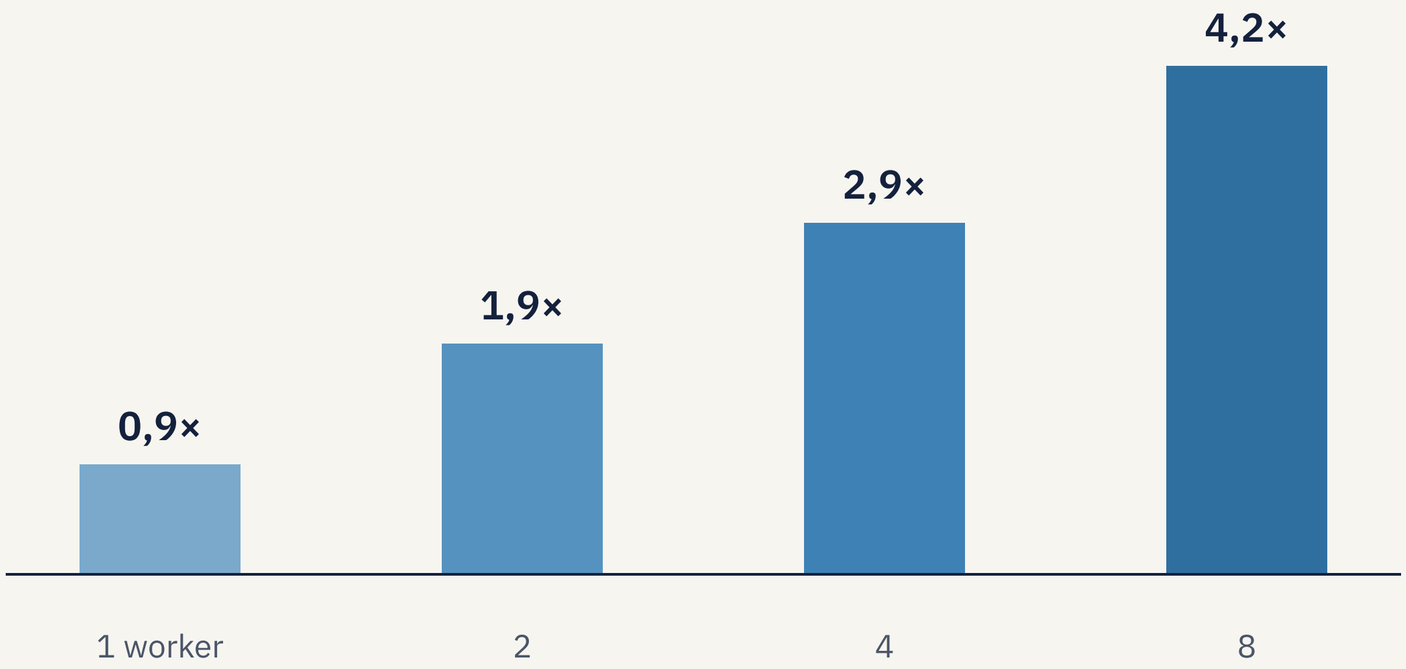
Herramienta	Para qué	Cómo
Valor inmutable	compartir sin coordinar	se pasa el puntero; no puede cambiar
Actor	serializar lógica arbitraria sobre un estado	asActor; un mensaje por vez
Atom	una celda que muchos actualizan sin cuello de botella	value:ifCurrent: (CAS) y swap: con reintento

```
total := Atom on: 0.
total swap: [:v | v + 1].
```

El CAS compara identidad de instantáneas inmutables: sin ABA. El bloque de swap: tiene que ser puro, porque puede correr más de una vez.

EJERCICIO 10 · CUÁNTO ESCALA

El mismo cálculo en 12 actores: 4,2× con 8 workers



Primos hasta 120 000, un hilo contra 12 actores, con 1, 2, 4 y 8 workers (PROTOST_WORKERS).

Sin lock global, el techo viene de otro lado: cada escritura publica una instantánea, el GC tiene un solo marcador y el pool no pasa de 8.

Si el cálculo asigna (ejercicio 4), cuatro actores rinden 1,8×.

Medido el 29/10 con protoST 0.7.0 en Windows 11, Intel Core Ultra 7 155U (12 núcleos híbridos), una corrida por punto · 42

PARA DISCUTIR · ACTORES Y OTROS LENGUAJES

1. ¿Alcanzan los actores como proxies, o Smalltalk necesita Process?
2. ¿Debería un ^ cruzar actores? ¿Debería aplanarse un Future de un Future?
3. ¿Qué gana y qué pierde Smalltalk al compartir objetos sin copia con Python o JavaScript?
4. ¿Qué programa suyo les gustaría ver corriendo en ocho núcleos?

LABORATORIO 2 · 35 MINUTOS

La VM, los actores y los módulos

`05-bytecode.st`

capturas, inlining, IVAR_GROUP con --disasm

`06-retorno-no-local.st`

^ por do:, home muerto, ensure:, otro actor

`07-excepciones.st`

resume:, retry, ifCurtailed:, outer

`08-actores.st`

proxy, deadlock, thenDo:, futures anidados, errores

`09-modulos.st`

Import from:, caché, privadas; lectura guiada de interop

`10-escalar.st`

PROTOST_WORKERS = 1, 2, 4, 8

Si terminan: las líneas abiertas de la guía (outer, el ^ entre actores, un inline cache, futures anidados).

LO QUE NO SÉ

No sé hasta dónde puede llegar esto

- si escala más allá de ocho núcleos, en máquinas grandes
- cómo se compara con Cuis o VAST en un hilo: nunca lo medimos
- si el costo de memoria (64 bytes por elemento, sin compactar) aguanta programas reales
- cuáles de las faltas son esenciales al modelo y cuáles son trabajo pendiente
- qué pasa con programas grandes, escritos por gente que no lo diseñó

Cada una de estas es una investigación, y ninguna la puedo hacer bien solo.

DÓNDE PODRÍAN METERSE

Líneas abiertas, cada una con un punto de partida

Línea	Por dónde empezar
Contextos y depuración	materializar frames bajo demanda; snapshotFrames de los actores como precedente
Envío más rápido	inline caches guardados por instantánea; benchmarks/compare_builds.py para medir
GC	generación vieja sin barrera; percentiles de pausa con PROTOCORE_HEAP_TRACE
Semántica	outer, become: de identidades mutables, futures anidados
Comparación	los mismos benchmarks en Cuis y VAST, en sus máquinas
Programas reales	uno suyo, en ocho núcleos, y un issue con lo que sorprenda

`github.com/gamarino/protoST · MIT · la guía de trabajo está en el repositorio`